

Next, we call `pygame.Rect()` to create a rectangle ❹. Defining a rectangle requires four parameters—an x-coordinate, a y-coordinate, a width, and a height, in that order:

```
<rectObject> = pygame.Rect(<x>, <y>, <width>, <height>)
```

This returns a pygame rectangle object, or `rect`. We'll use the rectangle of the ball in the processing of events.

We also add code to check if the user clicked the mouse. As mentioned, a mouse click is actually made up of two different events: a mouse down event and a mouse up event. Since the mouse up event is typically used to signal activation, we'll only look for that event here. This event is signaled by a new `event.type` value of `pygame.MOUSEBUTTONDOWN` ❺. When we find that a mouse up event has occurred, we'll then check to see if the location where the user clicked was inside the current rectangle of the ball.

When pygame detects that an event has happened, it builds an event object containing a lot of data. In this case, we only care about the x- and y-coordinates where the event happened. We retrieve the (x, y) position of the click using `event.pos`, which provides a tuple of two values.

NOTE

If we need to separate the x- and y-coordinates of the click, we can unpack the tuple and store the values into two variables like this:

```
mouseX, mouseY = event.pos
```

Now we check to see if the event happened inside the rectangle of the ball using `collidepoint()` ❻, whose syntax is:

```
<booleanVariable> = <someRectangle>.collidepoint(<someXYLocation>)
```

The method returns a Boolean `True` if the given point is inside the rectangle. If the user has clicked the ball, we randomly select new values for `ballX` and `ballY`. We use those values to create a new rectangle for the ball at the new random location.